Spring break assignment 2025-2026

Objective

Combining the topics of the past weeks

Controlling new components

Use classes and libraries that you have created in the past few weeks

Understanding data communication protocols

Using Bit Operations

Practicing the subject matter (preparation for the exam)

Introduction

This assignment is an ideal way to review and practice the topics of the Sensors & Interfacing Lab. It brings together concepts from the past few weeks, letting you see how well you've mastered the material. Students who have done well on previous assignments will likely find this assignment quite manageable. For others, it's the perfect time to catch up. Work on this assignment independently – it's excellent preparation for the exam.

Remember, the Sensors & Interfacing module counts for 20% of your final grade, split evenly across the two exam periods. This assignment is a major component of that continuous evaluation. Submit your assignment to Leho and be ready to explain it during the first lab after spring break. Explaining your work is how you earn your points. If you're absent, make sure you have an official excuse (via iBamaflex) and contact me to reschedule.

The assignment covers several topics. Even if you get stuck on one, you can still move on to others and earn points. Think of it as a chance to show what you know! Let me know if you have any questions.

Important Notes: Limitations and Requirements

We've asked you to complete lab assignments and prepare presentations in recent weeks. The idea is for you to build on that work for this assignment. Using external libraries that we haven't covered in class can be tricky, and unfortunately, it will result in 0 points (or even negative points) on those parts of the assignment.

Only for the I2C bus (LCD and ADC) you may use the smbus library, and of course the RPi.GPIO library for GPIO operations.

Also “standard” python libraries like os, subprocess, ..., sleep can be used.

You may also use the pip libraries related to Bluetooth

To summarize:

- Using basic string operations instead of bit operations => no points for that specific topic.
- Using external libraries (beyond those mentioned above) => no points for that topic, and possibly negative points.
- Not being able to explain your work verbally => 0 points for the entire assignment (though you'll still benefit from the practice).
- Submitting after the Easter holiday deadline => 0 points (but again, the practice is valuable).

Planning

It's a good idea to read through this assignment carefully before the Easter holidays. If anything seems unclear, feel free to reach out via Teams for an explanation. I'll do my best to answer your questions promptly, but I can't promise a 5-minute response time.

Since you just learned about controlling the LCD display, I'll try to address questions about that topic even during the break, depending on my availability, but only during the first week.

Submit

On leho there is an assignment where you can submit a zip file.

The submission file must contain all your code (main code + classes) + photo of your electronics (RGB Led, LCD Display) with some custom text on the LCD (from screen 5) with your student ID visible

Please submit your assignment before Thursday 23 13:30 / Friday 24 April 08:30. The completed assignment will have to be explained and demonstrated in the first week after the spring break (i.e. the week starting with April 20)

(during the lab after the explanation).

Please be aware that the schedule has changed, the lab for Group 02 is moved to Friday, Group 01 has (individual labs) both on Thursday and Friday (because the week after 1 of the classes is canceled because of the May first holiday)

Supplies

1) Raspberry Pi
2) Project board with external components:
    a. RGB Led
    b. LCD

Topics

1) Joystick with ADC

Read, by polling with a small delay, the analog values of the X and Y axes of the joystick.

Use a callback method to register presses on the built-in button from the joystick. Each time the joystick is pressed, print "The joystick has been pressed {number} times!" to the console. Count the "number" 1 up every time.

2) LCD display

You have to create multiple "screens" on the LCD

You can toggle between the screens with the button included in the joystick (GPIO7)

Screen1 > screen2 > screen3 > screen4 > screen5 > screen6 > screen1 > screen2 > screen3 > ...

Screen1:

Show your ipv4 address of the WiFi and LAN adapter

> 1 per line, specify which IP is from wich adapter
(run system command in python, capture output, filter(?) and display on the LCD)

Via the command console we can retrieve the IP addresses in the following way: via the instruction "ip a" we can obtain the IP addresses of the pi. We get the wlan0 and eth0 address back.

Here we can use string operations to figure out which adapter has what IP and display them on the LCD.

In Python we can connect via the subprocess module via various input and output pipes.

https://docs.python.org/3/library/subprocess.html#subprocess.check_output

With the **check_output** method, we can execute linux commands and get the output as a return value.

In the ips return value you will find the IP addresses. Take them out nicely and display them on the LCD screen.

Optionally, you could filter out the right IP address by using bash command:

```
pibeeckm@pibeeckm-Latitude-5520:~$ ifconfig wlp0s20f3 | grep 'inet ' | awk '{print $2}' | cut -d: -f2
192.168.1.34
```

You should replace wlp0s20f3 with the name of the network adapter you want the IP from (probably eth0 or wlan0)

Screen2:

Read the state of the 4 buttons and combine (bits!!) them into a nibble

On the first line of the LCD print out the binary value, aligned left and the hexadecimal value aligned right

On the second line, print the corresponding decimal value, aligned right

When a new combination of buttons is pressed, the LCD will update

e.g.:

```
0b1111        0xf
               15
```

(binary aligned left on first line; hex aligned right on first line; dec aligned right on second line)

Screen3:

Continuously read the value of the X-axes of the joystick

On the first line of the LCD, give a visual representation (as if it were a LED Bar Graph) of this value
On the second line of the LCD print "VRX => {val}" where {val} represents the decimal value

Update in real-time

Screen4:

Same as screen3, but for Y-axes (and print "VRY" instead of "VRX")

Screen5:

MPU accelero data

first line, aligned left: Accelero X data (in g)

first line, aligned right: Accelero Y data (in g)

second line, aligned left: Accelero Z data (in g)

second line, aligned right: Combined (in g)

use 2 digits (0.01)

Screen6:

Create a GATT UART BLE service on the RPi

When you arrive at this screen, print "Send a message via BLE UART" over the 2 lines

When a new message arrives, display this message on the LCD.

3) RGB LED

Convert the scale of the joystick to a percentage and pass it to a function that controls the RGB LED.

Connect the RGB LED (be mindful of 5V pin!). Make sure that the X-axis of the joystick matches one color (segment of the RGB), the Y-axis matches a second color. The third color is the average of the first two colors.